

# Language Table exp01

Diagnostic Report — frozen-Qwen + LoRA adapter + DiT VLA

10% subset · 18,102 episodes · 13,268 unique instructions · best val 0.4741 (ep 145)

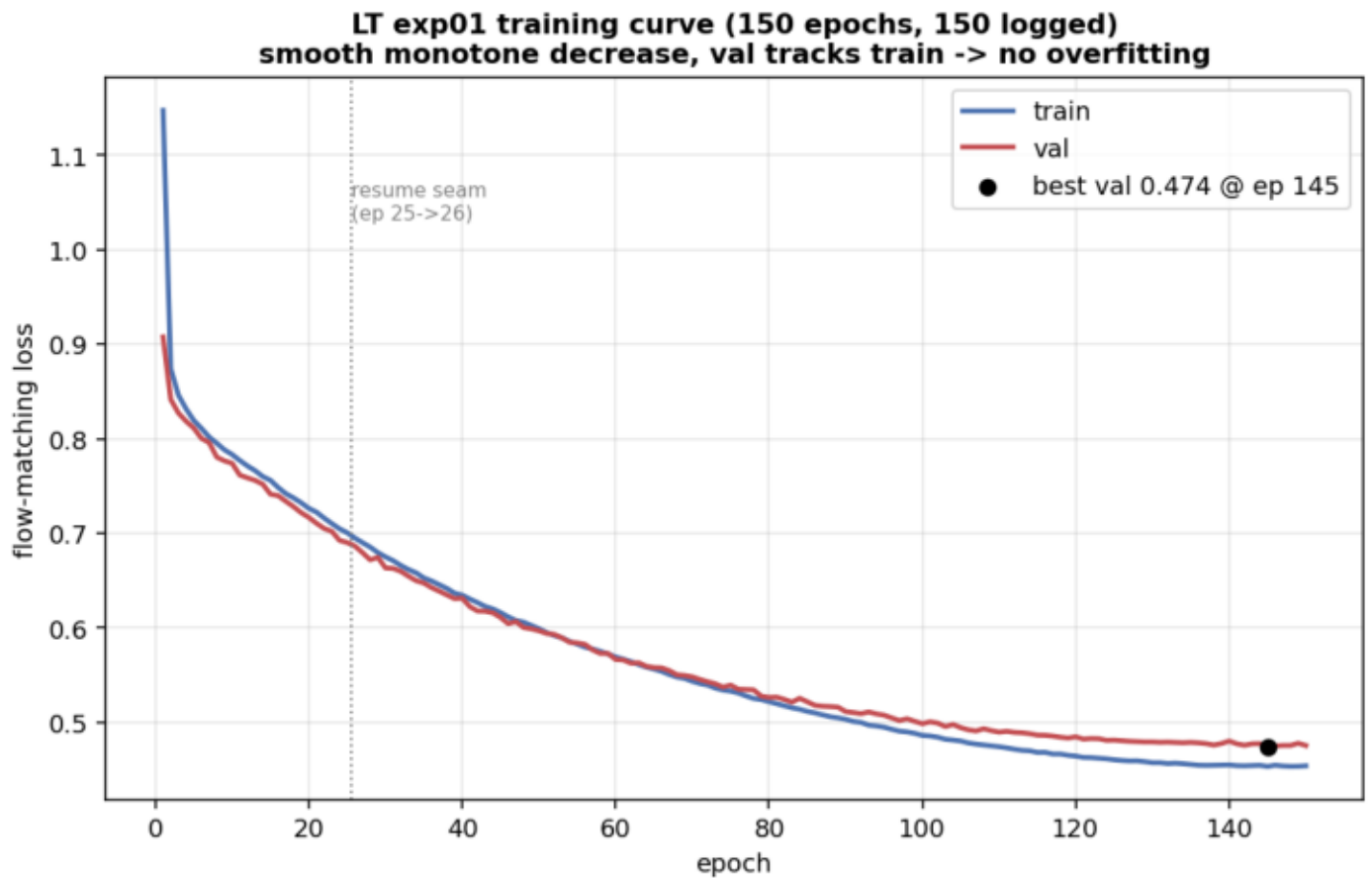
Central question: the model clearly USES language, yet block2block success is only 22%. Is that a language problem or a control problem?

## What the 11 plots establish, in one line each:

- Training is converged and not overfit → under-training ruled out (plot 1)
- Language is used causally → 2.1–2.6× action shift, 92%→36% on a wrong instruction (plots 5–6)
- The mechanism is real → text tokens ~2× over-attended; heads spatially specialise (plots 7, 9)
- The gap is manipulation → point 92% vs block2block 22%, same grounding (plot 2)
- Failure is distance-limited → 52% stall far; SR 35% short vs 5% long (plots 3–4)
- Perception is coarse → diffuse frozen visual features compound long pushes (plot 8)
- Behaviour is purposeful → beats a random policy on every test (plots 10–11)

**Conclusion: language grounding is solved; the ceiling is long-horizon closed-loop pushing with coarse visual feedback — not language understanding.**

# 1 · Training curve



## What this plot is for

Gate-check that the model trained properly, so the modest 22% task success can't be blamed on bad optimisation. This is the 'is it even trained?' question we must answer before interpreting any behaviour.

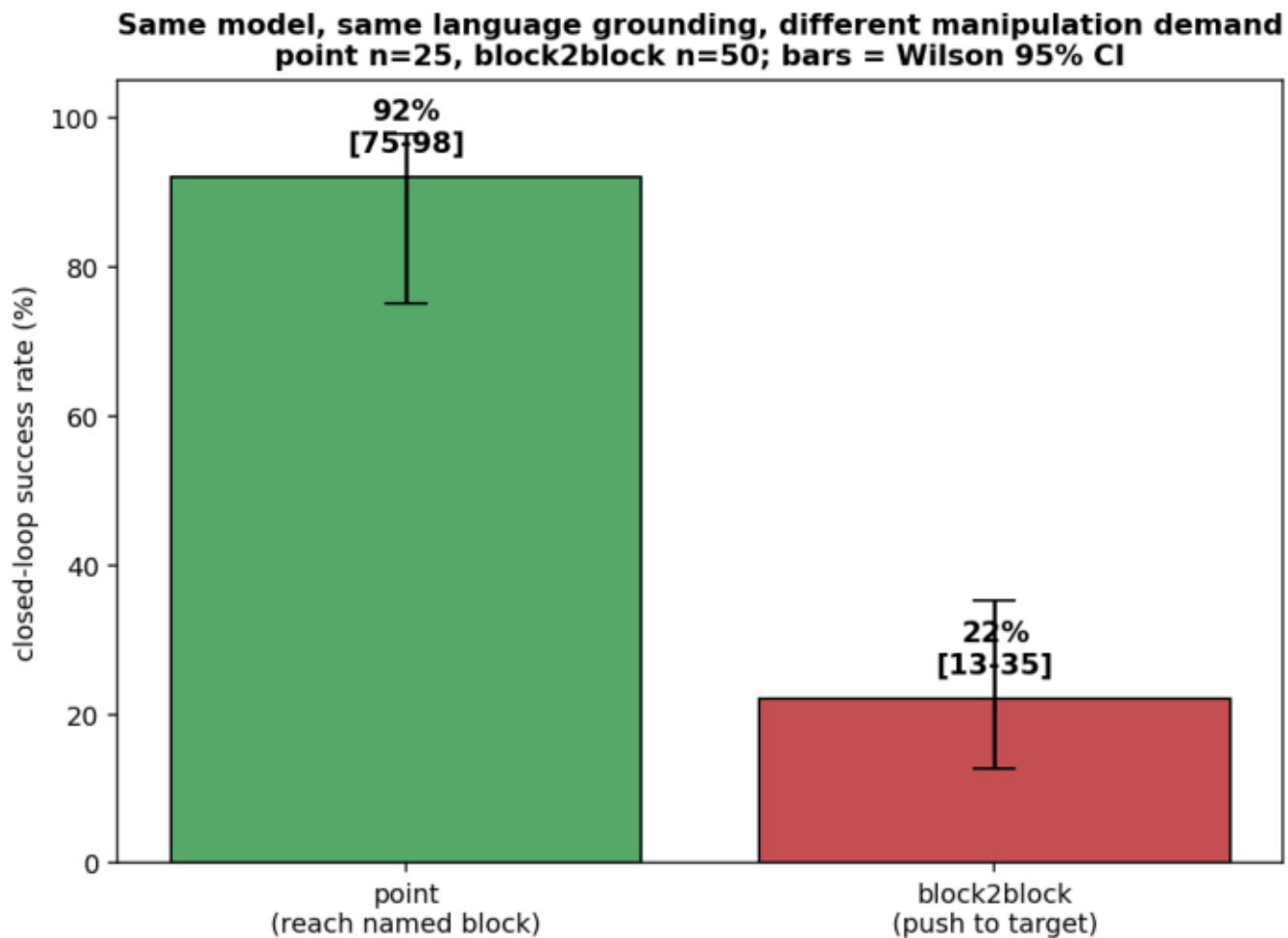
## How it was made

Parse per-epoch train/val flow-matching loss from the two pipeline logs (spanning the crash-resume seam at epoch 25), plot both against epoch, and mark the best-validation checkpoint (val 0.4741 @ epoch 145).

## What it tells us → next step

Smooth monotone descent; validation tracks training with a negligible gap and no late-epoch divergence → the model converged and did NOT overfit (13,268 unique instructions make memorisation impossible). UNDER-TRAINING IS RULED OUT. Next step: stop blaming training and look downstream — at grounding and control.

## 2 · Success rate — point vs block2block



### What this plot is for

Localise the bottleneck with a controlled comparison: two tasks that share the SAME model, checkpoint, and language grounding, differing only in how much manipulation they demand.

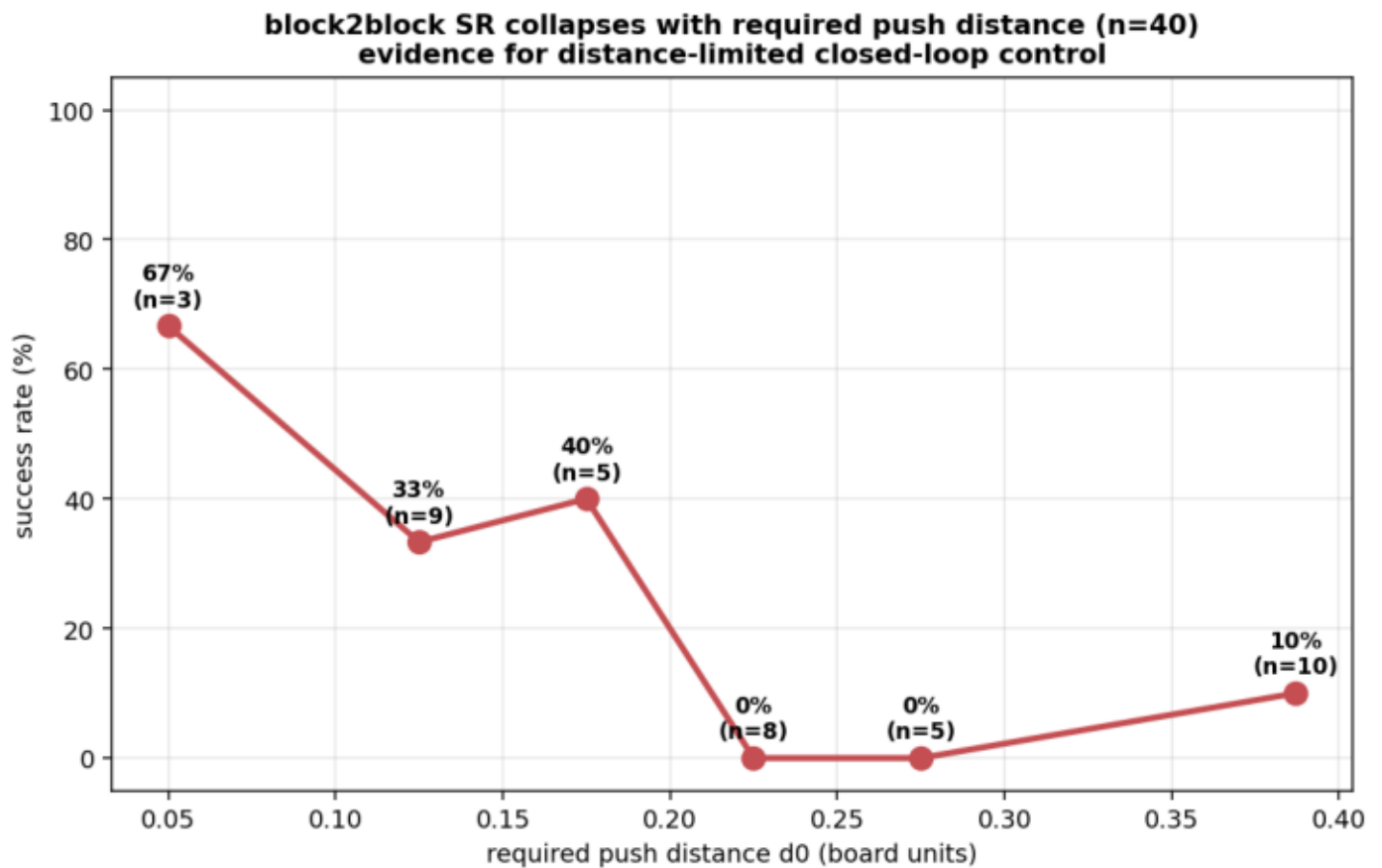
### How it was made

Closed-loop sim rollouts. Point SR from the language-effect run (n=25), block2block from the standard eval (n=50). Error bars are Wilson 95% CIs.

### What it tells us → next step

92% (point) vs 22% (block2block), with non-overlapping CIs. A short reach to a named block almost always works; pushing a block across the board usually fails. Because everything upstream is identical, the gap IS the manipulation demand — GROUNDING IS NOT THE BOTTLENECK. Next step: characterise the manipulation failure.

### 3 · Success rate vs required push distance



#### What this plot is for

Test the most likely manipulation culprit: does block2block fail because of HOW FAR the block has to travel?

#### How it was made

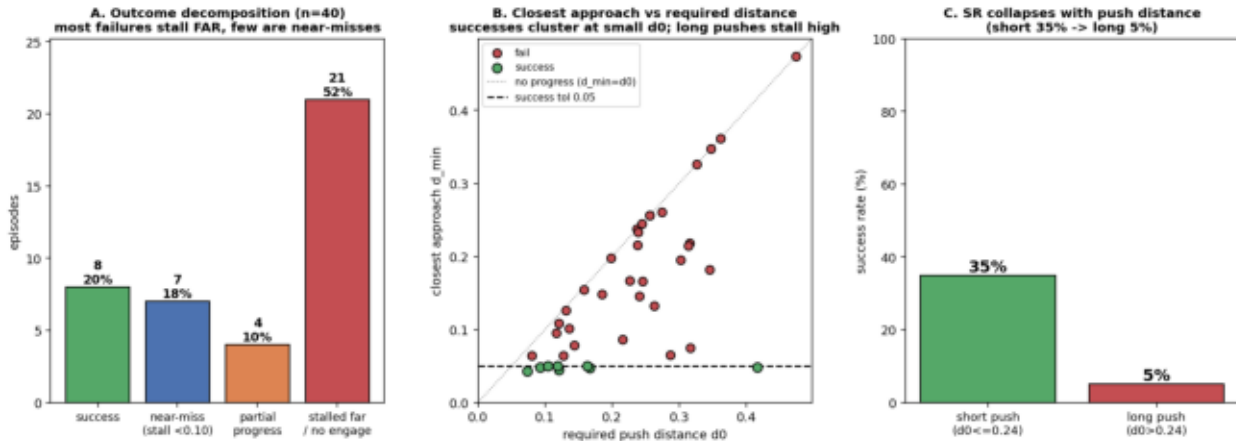
From 40 logged rollouts, bin each episode by its initial start→target distance  $d_0$  and compute the success rate within each bin.

#### What it tells us → next step

SR collapses past  $d_0 \approx 0.20$  (67% at the shortest distances → 0% in the mid-range). There is a hard capability cliff: the policy initiates the right push but cannot sustain it over distance — DISTANCE-LIMITED CLOSED-LOOP CONTROL. Next step: confirm with a full failure-mode decomposition (small bins here are noisy).

## 4 · Failure-mode decomposition

Why is LT exp01 SR only 20%? Grounding works — the correct block is pushed toward the target in 62% of episodes — but closed-loop control is DISTANCE-LIMITED: short pushes succeed 35%, long pushes only 5%; most failures stall far short of the 0.05 goal (median  $d_{\min}=0.17$ ). Bottleneck = sustained long-range pushing, not language understanding.



### What this plot is for

Separate the competing explanations for failure: wrong target (grounding), near-miss precision (fine placement), or stalling far short (long-range control)?

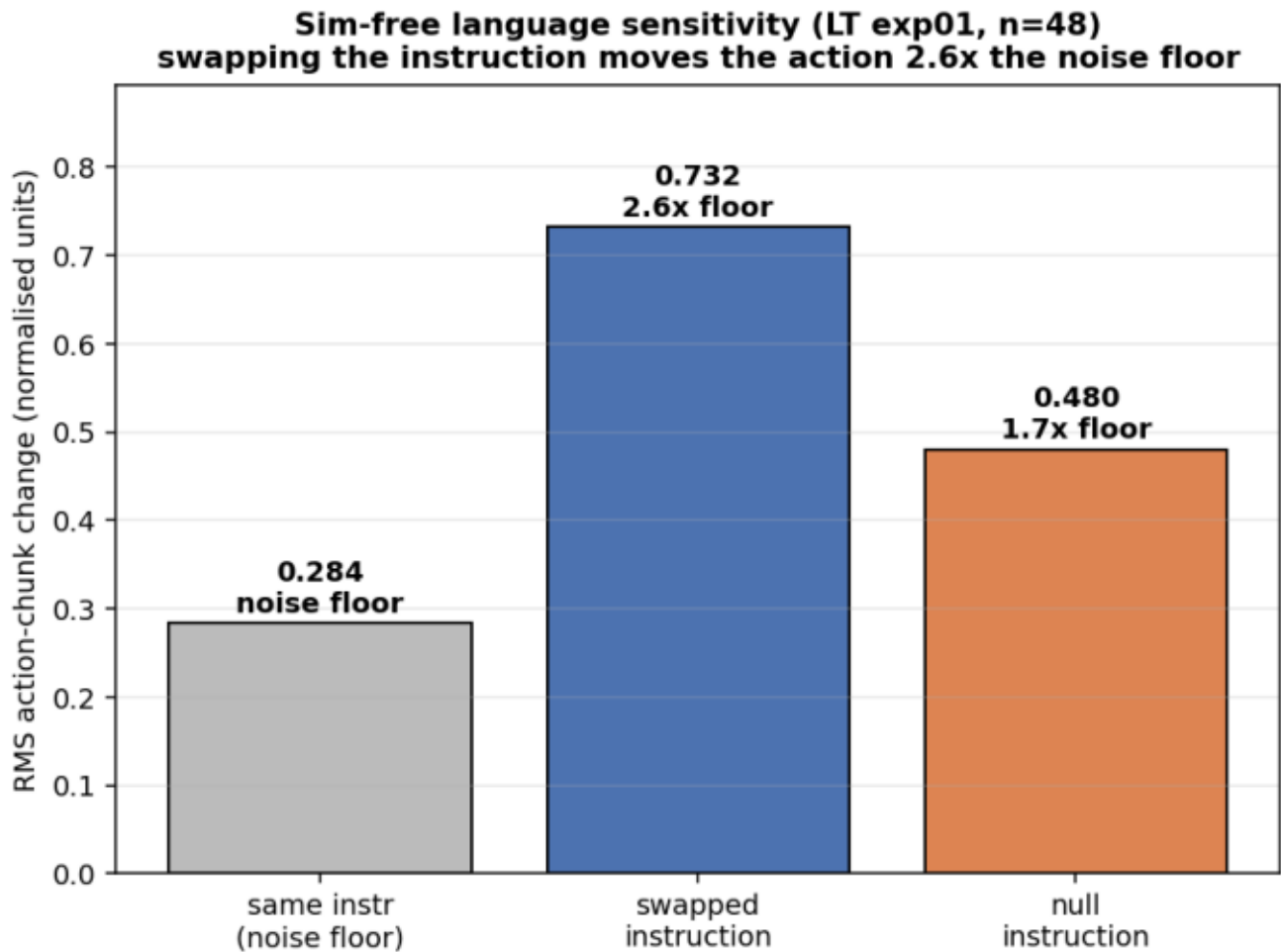
### How it was made

Replicate the exact eval rollout but log the start→target distance every step; categorise each episode (success / near-miss <0.10 / partial / stalled-far) and plot the outcome mix, a  $d_0$ -vs-closest-approach scatter, and SR for short vs long pushes.

### What it tells us → next step

The correct block is pushed toward the target in 62% of episodes (grounding works), yet 52% STALL FAR (median closest approach 0.174 vs the 0.05 goal) and SR is 35% short vs 5% long. Failures are not precision near-misses — they are long-range stalls. Next step: the lever is motor persistence + perception over distance, not language.

## 5 · Sim-free language sensitivity



### What this plot is for

Cheaply (no sim) establish whether the per-episode instruction CAUSALLY changes the predicted action — i.e. is the architecture language-adaptive at all?

### How it was made

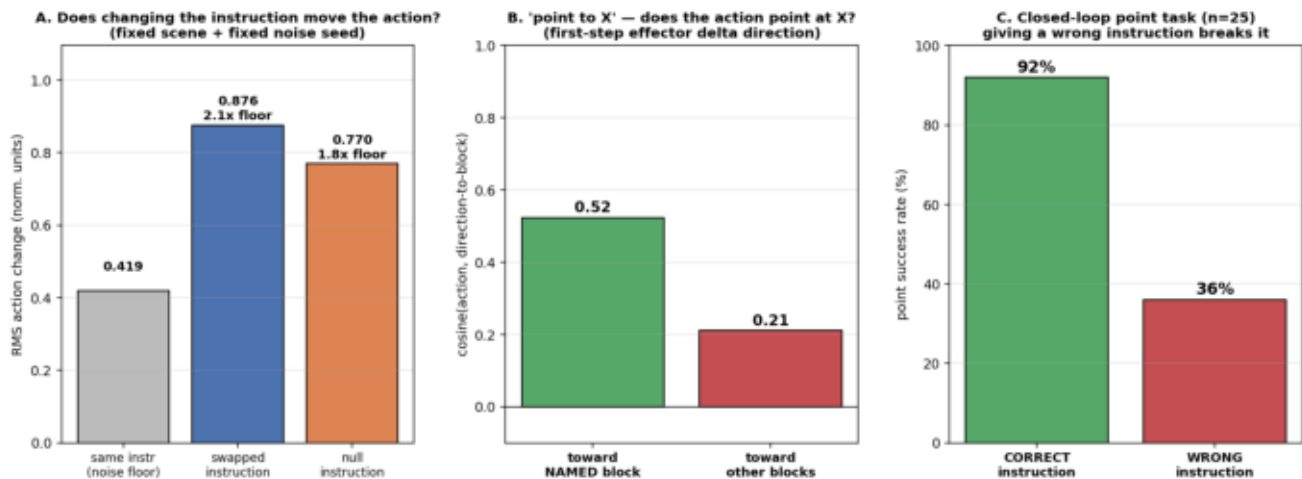
On 48 real dataset frames, hold the scene and the flow-matching noise seed fixed, then measure how far the predicted action chunk moves when we SWAP the instruction or NULL it, relative to the same-instruction sampling-noise floor.

### What it tells us → next step

Swapping moves the action 2.6x the noise floor; nulling 1.7x. The instruction has a strong causal effect on the action — STRONG LANGUAGE ADAPTABILITY, confirmed without the sim. Next step: verify the effect in the live closed loop and inspect its mechanism.

## 6 · Language effect (closed-loop, 3 ways)

How much does the instruction drive the model's decision?  
Swap moves the action 2.1x the noise floor · the action aims at the NAMED block (cos 0.52 vs 0.21) · a WRONG instruction drops point SR 92%→36%  
Language has a strong, causal effect on WHAT the model does; the weakness is long-range pushing, not language use.



### What this plot is for

Confirm the language effect causally in the live environment, from three angles, so it can't be an artefact of the offline probe.

### How it was made

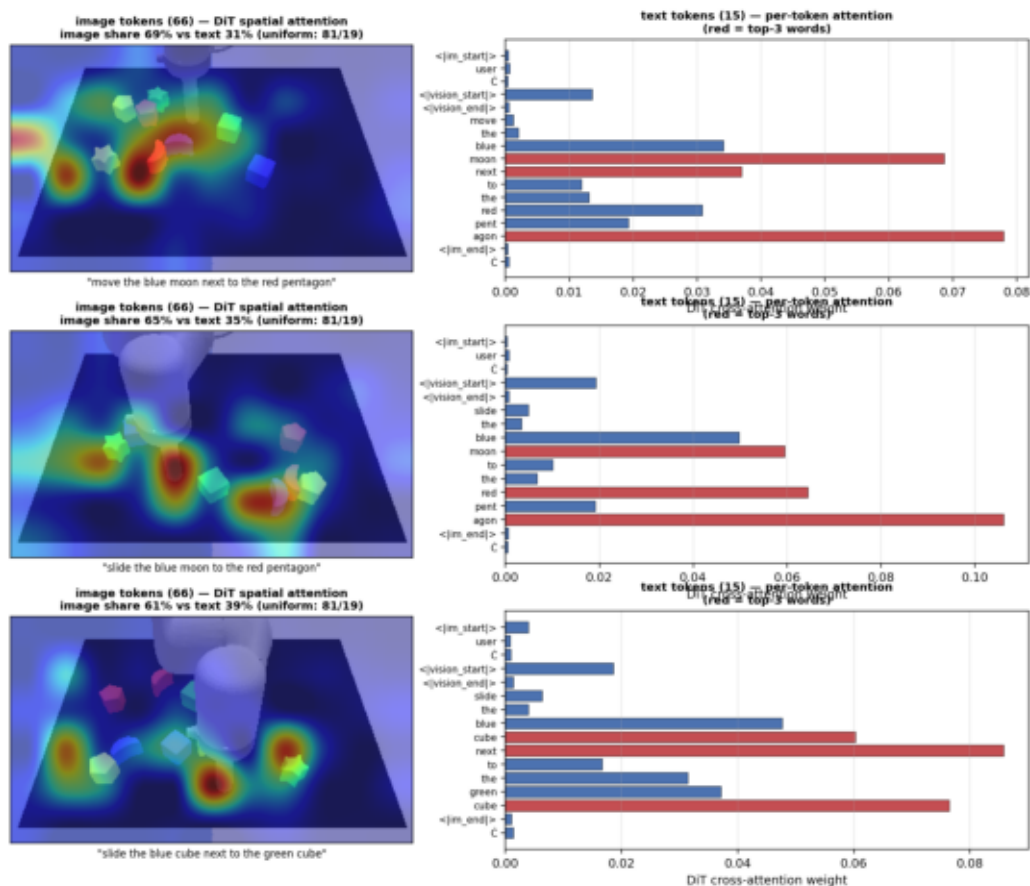
(A) action divergence vs the noise floor; (B) for 'point to X', the cosine of the first-step effector delta with the direction to the NAMED block vs OTHER blocks; (C) closed-loop point SR with the CORRECT instruction vs a WRONG one.

### What it tells us → next step

Action shifts 2.1× the floor; the move aims at the named block (cos 0.52 vs 0.21); correct-instruction SR 92% vs wrong-instruction 36% — a 56-point causal drop from language alone. The model genuinely follows the command; the weakness is long-range pushing. Next step: open the box and see HOW the instruction enters the action.

## 7 · DiT cross-attention (image + text)

How the DiT action decoder attends to EACH token (cross-attention, averaged over heads / blocks / action steps)  
Left: attention over the 66 image patches. Right: attention over each decoded text token — which instruction words the action looks at.



### What this plot is for

Reveal the mechanism of grounding: does the action decoder actually READ the instruction tokens, and where does it attend on the image?

### How it was made

Capture the DiT cross-attention weights and average over heads, decoder blocks and action-query steps. Left: a 6×11 image-patch heatmap. Right: per-text-token attention, with the share of attention going to image vs text tokens.

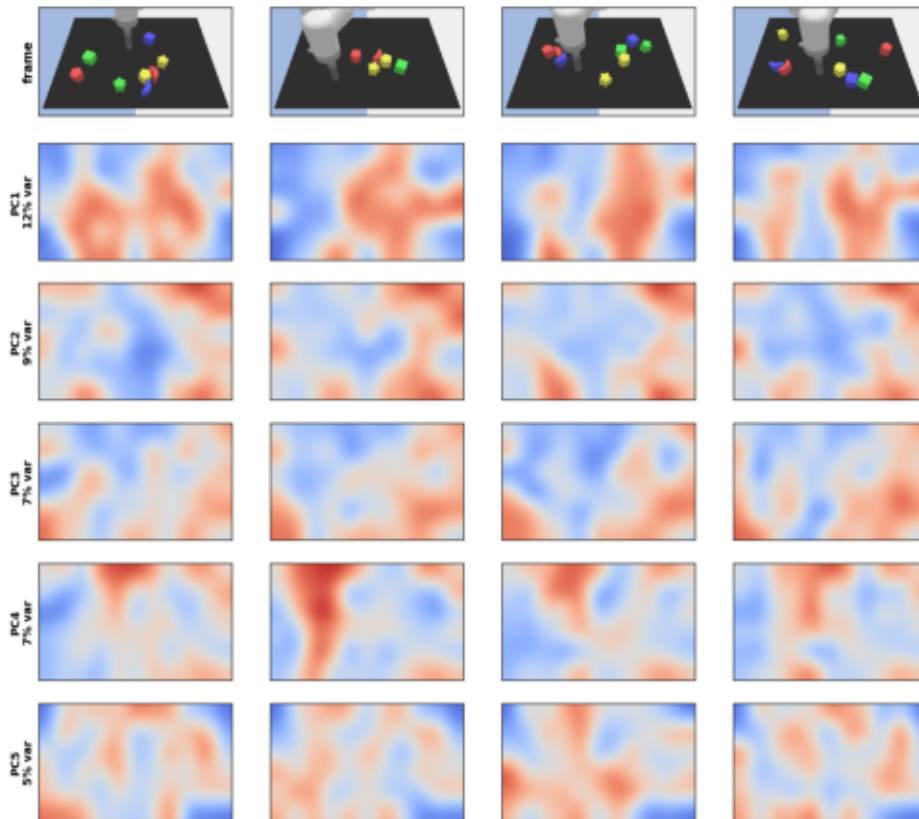
### What it tells us → next step

Text tokens receive 31–39% of attention while being only ~19% of the tokens — roughly 2× over-weighted — and object/colour/verb words dominate over filler words. The decoder actively reads the instruction. Next step: split the heads to understand the spatial structure.



## 8 · Frozen-Qwen visual feature PCA

What does the IMAGE embedding encode? — each principal component of the frozen Qwen patch features (6x11 grid) as a spatial map  
Instruction-independent (pure vision). Per-PC variance 12/9/7/7/5% — red/blue = +/- along that component. Look for components that light up on the arm, the table, or block regions.



### What this plot is for

Assess the QUALITY of the visual features the policy must localise blocks from — the model never receives block coordinates, only the image, so this is its entire spatial signal.

### How it was made

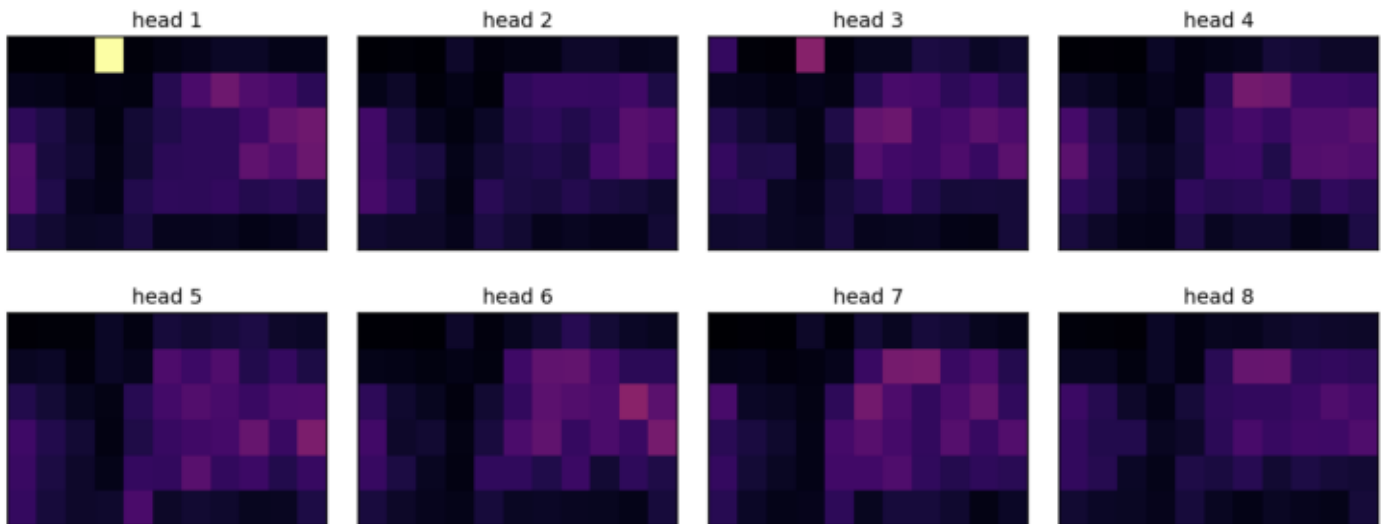
Run PCA over the 66 frozen Qwen image-patch features on live frames and map the top-5 principal components spatially across the board.

### What it tells us → next step

Features are diffuse and low-variance (PC1 only ~12%, broad gradients, no crisp per-block separation). Coarse localisation is enough to REACH a block (point 92%) but marginal for the precise, sustained contact a long push needs. Next step: a domain-adapted / fine-tuned visual encoder is the highest-value perception lever.

## 9 · Per-head DiT attention

**DiT cross-attention per HEAD over the 6x11 image grid (LT exp01, n=24 frames)**  
**each head specialises on a different board region; the per-token figure averages these 8 maps**



---

### What this plot is for

Explain why the averaged attention map looks 'distributed': is the model confused, or is the attention structured across heads?

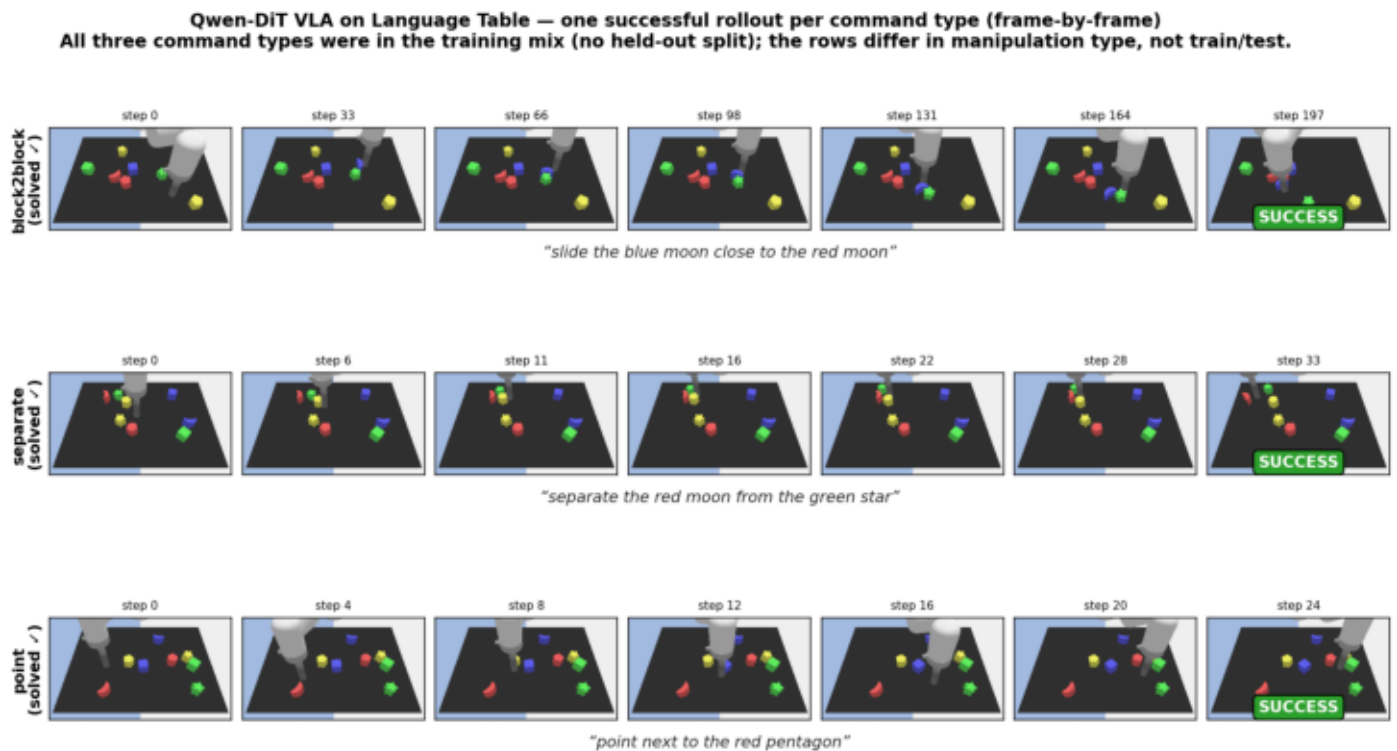
### How it was made

Monkey-patch the cross-attention with `average_attn_weights=False`, collect each of the 8 heads' image-grid attention separately, and average over frames, blocks and query steps.

### What it tells us → next step

Each head specialises on a DIFFERENT board region — a learned division of labour, not noise. The averaged map looks smooth only because it superimposes 8 focused heads. Next step: a 'target-tracking' head losing its lock during a long push is a plausible micro-mechanism for the stalls seen in plot 4.

# 10 · Qualitative rollouts (filmstrip)



## What this plot is for

See, frame by frame, that the policy genuinely executes the commanded manipulation — a human-readable sanity check behind the aggregate numbers.

## How it was made

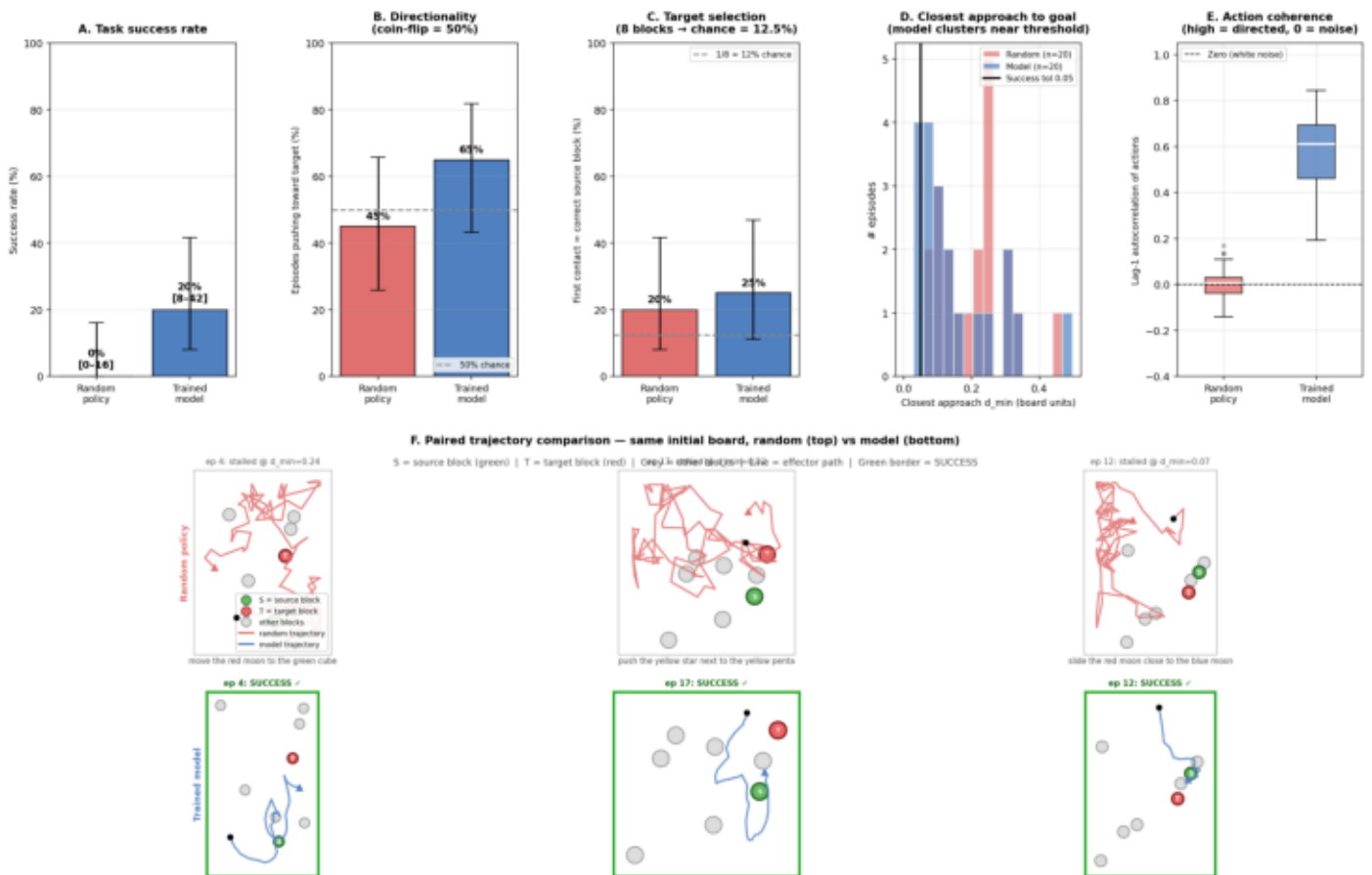
Roll out until one SUCCESS per command type (block2block / separate / point), then sample 7 evenly-spaced frames with the instruction caption. NB: this run trained on all command types (no held-out split) — the rows differ in manipulation type, not train/test.

## What it tells us → next step

Correct target selection, correct spatial relation ('towards', 'from', 'next to'), and task completion are all visible; point is by far the fastest (short-horizon). Qualitative confirmation of grounding AND execution capability. Next step: supports the synthesis.

# 11 · Anti-random control (is it just flailing?)

Is the model just randomly moving? — Four independent tests say NO  
SR 0% → 20% · directionality, target selection, and action coherence all far above chance · trajectories are directed, not scattered



## What this plot is for

Rule out the deflationary alternative: that the 22% successes are random flailing that occasionally bumps two blocks together, rather than purposeful control.

## How it was made

Paired rollouts of a RANDOM policy vs the trained model on IDENTICAL episode initialisations (same seed). Four independent tests — success rate, directionality, first-contact target selection, and lag-1 action autocorrelation — plus matched trajectory overlays on the same boards.

## What it tells us → next step

Random 0% vs model 20% SR; model action autocorrelation 0.58 vs random 0.01 (directed motion vs white noise); the model pushes toward the target above the 50% coin-flip; its trajectories are directed, not scattered. THE BEHAVIOUR IS PURPOSEFUL AND LANGUAGE-CONDITIONED, not random. Next step: cements the 'control, not randomness' framing.

# Synthesis & recommended next experiments

## The evidence chain

1. Trained & converged (plot 1) — 22% is a real capability, not an optimisation artefact.
2. Language drives behaviour causally (plots 5–6) and the mechanism is visible (plots 7, 9).
3. Holding grounding fixed, point 92% vs block2block 22% isolates manipulation (plot 2).
4. The manipulation gap is distance-limited: 52% stall far, 35%→5% short→long (plots 3–4).
5. Coarse frozen visual features (plot 8) compound the fine end of the problem.
6. The policy beats a random baseline on every test (plots 10–11) — it is purposeful.

## The strong hypothesis

Language grounding is SOLVED for this architecture; the performance ceiling on Language Table is set by long-horizon closed-loop object pushing — sustaining a directed push over distance with coarse visual feedback — not by language understanding or final-centimetre precision.

## Two experiments that would harden this to 'confirmed'

- A. No-VLM / shuffled-token block2block (mirrors the ALOHA ablation): if SR and the 'toward-target' rate collapse, the 62% directed pushes are genuinely vision-grounded, not a motion prior.
- B. SR vs demonstration-episode length with larger n: directly tests whether long-horizon demos are under-fit — the mechanism behind the distance collapse — and adds the confidence bands the current 40-episode bins lack.

Highest-value fixes implied: fine-tune / domain-adapt the frozen visual encoder; add more long-horizon demos; consider a longer inference horizon or a recurrent object-tracking memory.